An Empirical-Based Security Benchmarking Framework for Container Confinement

by

Huzaifa Patel

A thesis submitted to the Faculty of Graduate and Postdoctoral Affairs
in partial fulfillment of the requirements for the degree of

Master
in
Computer Science

Carleton University
Ottawa, Ontario

© 2025 Huzaifa Patel

Nomenclature

VM virtual machine. 2

Chapter 1

Background

The existance of cloud computing can be attributed to virtualization, which is a technology that makes it possible for multiple different operating systems (OSs) to run concurrently, and in an isolated environment on the same hardware. How does virtualization achieve this? It makes use of a machines hardware to support the software that creates and manages virtual machines (VMs). A virtual machine (VM) is a virtual environment that provides the functionality of a physical computer by using its own virtual central processing unit (VCPU), memory, network interface, and storage. The software that creates and manages VMs is formally called a hypervisor or virtual machine monitor (VMM). The virtualization marketplace is comprised of four notable hypervisors, which are: (1) VMWare, (2) Xen, (3) Kernel-based Virtual Machine (KVM), and (4) Hyper-V. The operating system running a hypervisor is called the host OS, while the VM that uses the hypervisor is called the guest OS. To maintain consistency within this proposal, we will use the word "hypervisor" when referring to virtualization software. Hypervisors can be divided into two types, depending on where they are located on the machine: (1) type 1 and (2) type 2.

1.0.1 Type 1 Hypervisor

Type 1 hypervisors run directly on physical hardware to create, control, and manage VMs, and do not require support from the host OS. Type 1 hypervisors are also called native or bare-metal hypervisors. The first hypervisors, which IBM developed in the 1960s were type 1 hypervisors [?]. Examples of type 1 hypervisors include, but are not limited to Xen, KVM, VMware ESX, and Microsoft Hyper-V [?].

1.0.2 Type 2 Hypervisor

Type 2 hypervisors (also called hosted hypervisors) are installed on the OS (Windows, Linux, MacOS), similar to how computer programs are installed onto an OS. In other words, a type 2 hypervisor runs as a process on the host OS. Type 2 hypervisors abstract guest OSs from the host OS by introducing a third software layer above the hardware, as shown in figure 2.1. Examples of type 2 hypervisors include but are not limited to VMware Workstation, VirtualBox, and QEMU [?].

1.0.3 Problems With Type 1 & Type 2 Hypervisor Classifications

Although the definitions of type 1 and type 2 hypervisors are widely accepted, there are gray areas where the distinction between the two remain unclear. For instance, according to its creator RedHat, KVM is implemented and deployed using two Linux kernel modules that effectively convert the host Linux OS into a type-1 hypervisor [?]. At the same time, KVM can be categorized as a type 2 hypervisor because KVM VM's are standard Linux processes that are competing with other Linux processes for CPU time given by the Linux kernel's native CPU scheduler [?]. Due to disagreements and vagueness in the definitions of type 1 and type 2 hypervisors, two new classifications were defined: (1) native hypervisors and (2) emulation hypervisors [?].

1.0.4 Native Hypervisor

Native hypervisors are hypervisors that push VM guest instructions directly to the physical machines CPU, by using virtualization extensions like Intel VT-x. [**?**]. Examples of Native hypervisors include but are not limited to Xen, KVM, VMware ESX, and Microsoft HyperV.

1.0.5 Emulation Hypervisor

Emulation hypervisors are hypervisors that emulate every VM guest instruction using software virtualization [**?**]. Emulated guest instructions are very easy to trace because all guest VM instructions are trapped to the hypervisor. Examples of emulation hypervisors include but are not limited to QEMU, Bochs, early versions of VMware-Workstation, and VirtualBox [**?**].

1.0.6 Why do we have run Docker with sudo?

By default, Docker's daemon (dockerd) runs as root, and access to it is controlled through the Unix socket:

Copy Edit /var/run/docker.sock This socket is owned by root and typically part of the docker group. If your user isn't in the docker group, you'll need sudo to interact with it.

Why Docker Needs Elevated Privileges: Container Isolation Requires Kernel Features:

Managing cgroups, namespaces, mounting filesystems, setting up virtual networks—all require elevated permissions.

By Default, Docker Protects Access to Its Control API:

docker.sock is a privileged control interface that allows full system control (you can effectively become root via it).

1.0.7 YAMA

Yama is a Linux Security Module (LSM) that adds additional restrictions to the ptrace system call, which is commonly used for debugging and process tracing. While ptrace is a powerful tool for legitimate uses like debuggers and profilers, it can also be abused by malicious processes to interfere with or spy on other running processes. Yama was introduced to mitigate these risks by controlling and limiting which processes can use ptrace on others, providing a simple but effective hardening mechanism against certain types of attacks.

The core of Yama's functionality revolves around the /proc/sys/kernel/yama/ptrace_scope setting, which controls how restrictive the kernel is regarding ptrace access. The available levels range from 0 to 3, each enforcing progressively stricter controls. With ptrace_scope=0, the system behaves traditionally, allowing any process to ptrace another process owned by the same user if the target is marked as dumpable. At 1, only parent processes can trace their children, limiting the risk of arbitrary tracing. Level 2 requires a process to have CAP_SYS_PTRACE capabilities (effectively root-level permissions) to trace any process, and level 3 completely disables ptrace system-wide, locking down the system from process tracing even by privileged users.

Yama is especially valuable in multi-user systems or environments where processes run under the same user account but should remain isolated from one another, such as in containerized or sandboxed environments. While it is a relatively lightweight security module compared to more comprehensive systems like SELinux or AppArmor, it adds a meaningful layer of defense against local privilege escalation attacks. However, it's important to balance its restrictions with the need for debugging and profiling tools that rely on ptrace. In production environments focused on security, higher ptrace_scope values (such as 2 or 3) may be appropriate, while development systems typically keep it at 0 or 1 to allow standard debugging workflows.

Yama does not enforce its ‘ptrace_scope‘ restrictions when attempting to trace host processes from within a container because Yama's enforcement is limited to the current process visibility defined by PID namespaces. In most container environments, the containerized process runs in an isolated PID namespace and cannot see or interact with host processes unless explicitly configured using options like ‘–pid=host‘. Even when a container is started with ‘–pid=host‘, allowing it to see host processes, Yama primarily governs interactions within the same namespace context and is not designed to enforce security policies across different namespaces.

Additionally, for Yama to restrict tracing of host processes, the containerized process would not only need visibility of host PIDs but also require sufficient privileges, such as ‘CAP_SYS_PTRACE‘, in the host's user namespace. In practice, other security mechanisms like AppArmor, SELinux, and seccomp often prevent such privilege escalation before Yama's policies even apply. As a result, Yama's effectiveness is primarily confined to preventing ptrace interactions between processes that are already within the same visibility scope, rather than serving as a cross-namespace isolation mechanism. This makes Yama a useful tool for hardening processes within the same environment but not a comprehensive defense against container-to-host process tracing attempts.

The behavior of Yama is controlled by the `/proc/sys/kernel/yama/ptrace_scope` setting, which determines the level of restriction imposed on `ptrace` operations. The possible values are summarized below:

When Yama's ptrace_scope is set to 2, processes cannot freely attach to other unrelated processes, even if they have administrative capabilities or are running as root. In this mode, the ability to trace or debug a process must be explicitly granted by the target process itself or configured through system-level controls.

This permission can be granted using the prctl() system call with the PR_SET_PTRACER option, allowing a specific process to act as a tracer. Alternatively, the system ad-

Table 1.1: Yama `ptrace_scope` Settings

Value	Behavior
0	No additional restrictions. Processes can trace any other process they have permissions for.
1	Restricted to child processes only. This is the default setting in many distributions.
2	Restricted unless explicitly allowed via `/proc/[pid]/uid_map` or using `prctl()`.
3	No `ptrace` allowed at all, even for processes with `CAP_SYS_PTRACE`. This is the most restrictive setting.

ministrator can modify the global ptrace policy by lowering the ptrace_scope value in /proc/sys/kernel/yama/ptrace_scope. Without such explicit authorization, ptrace attempts will consistently fail, regardless of user privileges or capabilities. This design enforces stronger security boundaries by preventing unauthorized inspection or manipulation of processes, even by privileged users, unless the target process explicitly consents to being traced.

Yama is a Linux Security Module (LSM) that enforces additional restrictions on the `ptrace` system call and process debugging mechanisms. Its primary purpose is to reduce the attack surface for exploits that rely on process introspection and manipulation. By limiting the ability of processes to trace or interact with others, Yama enhances system security and makes privilege escalation attacks more difficult. This is particularly valuable in containerized environments and multi-user systems where process isolation is critical.

1.0.8 Kernel Implementation

Yama is implemented in the Linux kernel source file `security/yama/yama_lsm.c`. It integrates with the kernel's LSM framework by registering specific hooks that intercept and evaluate `ptrace` operations. During initialization, Yama registers its hooks using the following function:

```
void __init yama_add_hooks(void)
{
    pr_info("Yama: becoming mindful.\n");
    security_add_hooks(yama_hooks, ARRAY_SIZE(yama_hooks), "yama");
    yama_init_sysctl();
}
```

Here, `security_add_hooks` registers an array of hook functions defined in `yama_hooks`. These hooks include:

- `ptrace_access_check`
- `ptrace_traceme`
- `task_prctl`
- `task_free`

Each hook is associated with a specific kernel operation, allowing Yama to enforce its security policies at critical points.

1.0.9 Enforcement of `ptrace_scope`

The core of Yama's enforcement logic is implemented in the `yama_ptrace_access_check` function:

```
static int yama_ptrace_access_check(struct task_struct *child,
         unsigned int mode){
}
```

This function determines whether a `ptrace` operation should be permitted based on the current `ptrace_scope` value and the relationship between the tracer and tracee processes.

1.0.10 `ptrace_scope` Settings

Yama provides four levels of `ptrace` restrictions, configurable via `/proc/sys/kernel/yama/ptrace_`

These settings are summarized in Table 1.1.

Chapter 2

Evaluation

2.0.1 Purpose

The purpose of this evaluation is to empirically assess the effectiveness, reliability, and practical utility of Houdini as a framework for detecting security issues, misconfigurations, and enforcement inconsistencies in containerized environments.

2.0.2 Requirements / Goals:

- **Reproducibility:** Houdini should produce consistent and repeatable results when run across different machines and environments. Identical configurations and test inputs should yield the same outcomes.
- **Scalability:** Houdini should support scalable execution, allowing users to easily configure, queue, and execute large batches of tests/tricks across multiple container configurations or versions. The framework should be designed to accept input declaratively and automate the full evaluation without requiring manual intervention.
- **Performance:** Houdini should execute tests efficiently, minimizing runtime overhead while maintaining accuracy. The framework's performance should

allow it to be integrated into larger workflows, such as continuous integration pipelines, without introducing significant delays.

- **Security and Host Isolation:** Houdini must isolate tests from the host system to ensure safety during the execution of potentially dangerous tricks. By running tests inside a virtual machine, Houdini should prevent container escapes or exploits from impacting the host. It should maintain a secure boundary between the test environment and the underlying system.

- **Accuracy of Detection** Houdini should correctly distinguish between successful and failed security isolation. Test results should reflect the actual runtime behavior of the container under evaluation without false positives or negatives.

- **Flexibility Across Configurations** The framework should support a wide range of container settings, including different Linux capabilities, namespaces, AppArmor profiles, seccomp filters, and Docker flags. It should also accommodate various versions of Docker, runc, crun, and the Linux kernel.

- **Transparency and Debuggability** Test runs should be transparent and easy to interpret. Logs, intermediate outputs, and error messages should be informative enough to help users understand why a trick passed or failed.

- **Detect Configuration-Behavior Inconsistencies:** Evaluate whether containers behave as expected under different security configurations by empirically validating real-world runtime behavior. This helps identify mismatches between intended policy and actual enforcement.

- **Assess Cross-Version Consistency and Enforcement Drift:** Determine whether the same container configuration yields consistent behavior across different versions of Docker, container runtimes (e.g., runc, crun), and Linux kernels.

2.0.3 Methodology / How we will do that:

To evaluate the effectiveness and design properties of the Houdini framework, we conduct a series of controlled experiments targeting key aspects of its architecture and operation. Each experiment is designed to test a specific goal of the framework, including reproducibility, scalability, performance, and security isolation.

We structure our methodology as follows:

1. **Reproducibility Across Machines**

   Run the same trick and configuration on two different machines (e.g., different CPU architectures or hardware specs) and verify that the output is identical.

2. **Reproducibility Across Time with Version Pinning ✓**

   Re-run a pinned Docker, kernel, and runc configuration at a later time to ensure consistent results, demonstrating Houdini's ability to reproduce experiments reliably.

3. **Scalability via Bulk Execution**

   Execute a large batch of tricks (e.g., 32) across multiple configurations in one session. Measure runtime, memory usage, and failure rates to assess Houdini's batch execution capabilities.

4. **Performance Benchmarking ✓**

   Measure the end-to-end time it takes Houdini to execute a single trick, including VM boot, container launch, and result collection. Compare this to a manually executed baseline outside Houdini.

5. **Host Isolation Validation ✓**

   Run a deliberately malicious trick (e.g., attempting to write to host directories) and verify that the host remains secure, demonstrating that Houdini's use of VMs effectively isolates test environments.

6. **Fault Tolerance and Crash Recovery**

   Introduce failing or crashing tricks (e.g., segmentation faults, infinite loops) and confirm that Houdini logs the failure and continues running subsequent tests without interruption.

7. **Version Variability Support ✓**

   Create a test matrix across different combinations of Docker, runc, and kernel versions. Confirm that Houdini successfully builds and runs tricks across all variations.

8. **Extensibility Test ✓**

   Develop a new custom trick (e.g., testing for a capability or syscall) and integrate it into Houdini. Measure the effort and steps needed to add and run it without modifying the core logic.

9. **CI Integration Test**

   Integrate Houdini into a CI/CD system such as GitHub Actions or GitLab CI. Trigger it on container configuration changes and assess its stability and reporting in an automated pipeline.

2.0.4 What Did You Find Out?

TODO when I actually run the experiments, but for the last two goals I have some results:

- The goal **"Detect Configuration-Behavior Inconsistencies"** is directly supported by **Case Studies 2, 3, and 4**, where the same configurations sometimes led to unexpected or insecure behavior (e.g., host interaction without capabilities, port binding despite restrictions, and ptrace behavior differing by scope).

- The goal **"Assess Cross-Version Consistency and Enforcement Drift"** is supported by **Case Studies 1, 3, and 4**, where the same trick or config yielded different results across versions of Docker, runc, and Linux distributions.

- **Case Study 1 (CVE-21626):** Discovered that older versions of `runc` (e.g., rc1–rc4), not listed in the official CVE disclosure, were also vulnerable. This expanded the known scope of the vulnerability and revealed version-scoping inconsistencies.

- **Case Study 2 (Host Process Manipulation):** Identified the minimum set of privileges required for a container to successfully interact with host processes. Demonstrated that certain combinations of namespace sharing and UID matching can bypass expected isolation controls, even with reduced capabilities.

- **Case Study 3 (Docker Privileged Port Binding):** Found that containers could bind to privileged ports under newer Docker versions despite appearing unprivileged. This revealed misconfigurations and inconsistencies between intended policy and actual runtime behavior.

- **Case Study 4 (ptrace Enforcement Across Linux Distributions):** Empirically validated that ptrace enforcement behavior varies across Linux distributions due to differences in kernel versions and default Yama settings. Exposed enforcement drift[1] and platform-dependent inconsistencies in confinement guarantees.

[1] Drift refers to changes in security behavior over time due to image updates, runtime automation, or infrastructure modifications.

2.1 Reproducibility Across Time with Version Pinning

This experiment evaluates Houdini's ability to reliably reproduce test results over time using a version-pinned container environment. Reproducibility is a core requirement in security evaluation, particularly when comparing behavior across different kernel versions or runtime configurations. Even slight changes to binaries or system dependencies can lead to unexpected behavioral drift. To address this, Houdini provides a controlled execution framework where all critical components are locked to specific versions.

Version pinning refers to explicitly locking the versions of software components, such as Docker, runc, and the Linux kernel to prevent variation due to automatic updates, rebuilds, or environmental drift. By re-executing a simple, deterministic "Hello World" trick across time and validating both behavior and binary integrity, this experiment assesses whether Houdini can produce identical results when the test configuration is held constant.

2.1.1 Experimental Setup

Rather than relying on Buildroot's package infrastructure, we used an `overlayfs` directory to inject precompiled binaries of Docker and runc directly into the root filesystem. This approach allows full control over the environment and guarantees that only explicitly specified versions of components are used.

The following components were pinned in this experiment:

- **Docker:** 27.0.3, installed at `/usr/bin/docker` and `/usr/bin/dockerd`
- **runc:** 1.1.13, placed at `/usr/bin/runc`
- **Linux Kernel:** 6.0, compiled using Buildroot and booted via QEMU

To ensure binary-level consistency between runs, we calculated the SHA256 hashes of each component and stored them in a version manifest file. These hashes act as a cryptographic fingerprint to confirm that the binaries remain unchanged throughout the experiment:

- **Docker:** `898c247dd668357238d008fefcd87946d407352a4b06efed6c20b45a1c7175e3`
- **runc:** `479928392cfba3809f874f1d3f07c5207a9e91d747d083b5753a8332d60d60a1`
- **Kernel:** `3b0313064fd076cb698357ec4ba80436975ce2d2f6ac440a82cded0cdbbd31c7`

These values were re-verified at the time of re-execution using the `sha256sum` utility to confirm that no environmental drift had occurred.

2.1.2 Hello World Trick Logic

The `hello_world.sh` script is placed in `/usr/bin/` and included via the overlay. Its behavior is intentionally simple and deterministic: it checks for the presence of a file called `/run/trigger_file` inside the container. If the file exists, the script prints "Trick passed"; otherwise, it prints "Trick failed." This condition allows us to control the outcome while keeping the logic repeatable and easily verifiable.

Listing 2.1: Hello World Trick Script

```
#!/bin/sh
if [ -f /run/trigger_file ]; then
    echo "Trick passed"
else
    echo "Trick failed"
fi
```

This trick is chosen not to test system vulnerabilities, but rather to validate that Houdini can reliably detect environmental consistency and reproduce test output based solely on deterministic conditions.

2.1.3 Initial Execution (T0)

At time T0, the environment was constructed using the pinned versions of Docker, runc, and the Linux kernel. The container was run without creating the trigger file:

Listing 2.2: Running the Hello Trick in a Container

```
1 docker run --rm myimage /hello_world.sh
```

Expected Output: Trick failed

All binary hashes were verified against the manifest, confirming that the test environment matched the intended configuration exactly. The result was recorded and used as a baseline for later comparison.

2.1.4 Re-run Execution (T1)

At time T1, after a delay, the Buildroot image was rebuilt using the same configuration and the original overlay directory. This ensured that all injected binaries were identical to those used at T0.

We re-verified the Docker and runc binaries inside the running VM using the following commands:

```
sha256sum /usr/bin/docker
sha256sum /usr/bin/runc
sha256sum bzImage
```

Both values matched the entries in `version_manifest.txt`, confirming no binary drift. The container was again executed without creating the trigger file:

Listing 2.3: Running the Hello Trick in a Container

```
1 docker run --rm myimage /hello_world.sh
```

Output: Trick failed

To test the positive condition, the trigger file was created within the container, and the trick was re-run:

Listing 2.4: Running the Hello Trick in a Container

```
1 docker run --rm myimage sh -c "touch /run/trigger_file && /
    hello_world.sh"
```

Output: Trick passed

These results exactly matched the expected behavior, confirming that the environment, logic, and outcome were all stable and reproducible.

2.1.5 Analysis and Observations

This experiment illustrates that when the container runtime stack is fully controlled through version pinning, overlay injection, and hash verification, Houdini can achieve bit-for-bit reproducibility. Moreover, this approach enables rigorous detection of behavioral drift caused by even minor environmental differences.

The results also highlight how small changes in the runtime environment, such as default mount behavior, file permissions, or base filesystem differences, could affect trick outcomes if not properly controlled. By tying the trick's behavior to a simple file-based condition, we were able to simulate both pass and fail outcomes in a deterministic and easily verifiable way.

2.1.6 Conclusion

This reproducibility experiment confirms that Houdini provides a reliable framework for time-stable container testing when all components are version-pinned and verifiable. By injecting Docker and runc via `overlayfs`, verifying integrity with SHA256 hashes, and using deterministic logic inside the test container, Houdini ensures that security and behavioral tests yield consistent results across time. This makes Hou-

dini well-suited for regression analysis, longitudinal container studies, and real-world vulnerability tracking.

2.2 Host Isolation Validation

This experiment evaluates Houdini's ability to isolate test environments from the physical host machine. Since Houdini executes potentially malicious or privilege-escalating code inside containers, it is critical to ensure that these operations cannot impact the host system. To validate this, we deliberately execute a container trick that attempts to write to a host-accessible path. The goal is to verify that Houdini's use of QEMU-based virtual machines effectively prevents the container from escaping the VM boundary.

2.2.1 Threat Model

The threat model for this evaluation assumes that the attacker has full control over arbitrary code executed inside a container running within Houdini's virtual machine. This reflects a realistic scenario where untrusted or potentially malicious container workloads are tested to evaluate security boundaries. The attacker may attempt to exploit kernel vulnerabilities, access unintended resources, or perform file system operations that target the host. Typical actions include attempting to write to host-mounted directories, reading sensitive data, or interacting with other host processes.

However, the attacker's capabilities are strictly confined to the virtual machine. They have no access to the physical host system, nor are they granted elevated privileges beyond what is available inside the containerized guest. The container may share namespaces with the host, and the kernel it interacts with is the one booted inside the VM, not the host kernel.

While Houdini containers by default operate in isolated namespaces, Docker allows

optional sharing of specific namespaces with the host environment using flags such as –pid=host, –net=host, –ipc=host, or –uts=host. When enabled, these options allow the container to observe or interact with resources on the host namespace level, such as the full process table, network stack, or inter-process communication channels. In typical host environments, this would increase the attack surface and make lateral movement or privilege escalation more feasible.

However, in Houdini's design, these namespace-sharing configurations are restricted to the context of the virtual machine only. That is, even if a container is launched with –pid=host, it shares the PID namespace only with the VM, not the physical host. Likewise, shared network or IPC namespaces remain confined to the QEMU instance. This ensures that while Houdini supports evaluation of shared-namespace behavior, any unintended access or compromise remains sandboxed inside the virtual machine. The attacker cannot affect the physical host's kernel or global namespaces, and cannot persist beyond the lifecycle of the VM.

The goal of this threat model is to validate that Houdini can safely execute such container scenarios without compromising host integrity. Even if the attacker succeeds in exploiting the container or VM environment, the outer boundary provided by QEMU must prevent any direct impact on the host. This includes protection against file writes, process interference, or network communication with host resources outside the intended testing interface.

This evaluation, therefore, focuses on confirming that the default Houdini configuration—with no shared folders, no privileged mounts, and no host-level interaction—effectively isolates the testbed. Any data or control flow out of the VM must pass through carefully scoped and auditable channels such as the Flask-based result reporting interface.

2.2.2 Malicious Trick Description

We designed a trick to evaluate whether a container running inside Houdini's virtual machine can access the physical host system. This test simulates an escape attempt by trying to write to a file path that would typically point to a shared folder if one were configured.

Listing 2.5: Escape Trick: Simulated Write to Host-Mounted Path

```
#!/bin/sh
# Attempt to write to a path that would be host-mounted if shared
    folders were enabled
TOUCH_PATH="/mnt/host_root/hacked.txt"

echo "Attempting to write to possible host mount at $TOUCH_PATH"
echo "HACKED" > "$TOUCH_PATH" 2>/dev/null

# Try to detect if the write succeeded
if [ -f "$TOUCH_PATH" ]; then
    echo "Trick passed: Possible host write detected!"
else
    echo "Trick failed: No host write access."
fi

# Optional: Check if root of PID 1 differs (may indicate shared PID
    namespace)
echo "Probing /proc/1/root:"
ls /proc/1/root 2>/dev/null | head -n 5
```

2.2.3 Execution Inside the VM

The container was executed inside a QEMU-based Buildroot virtual machine that does not expose any host directories and uses no shared folders. The container was

started using the following command:

Listing 2.6: Running Escape Trick Inside VM

```
docker run --rm alpine /host_escape_trick.sh
```

Example Output:

```
Attempting to write to possible host mount at /mnt/host_root/hacked.txt
Trick failed: No host write access.
```

No file named `hacked.txt` appeared on the physical host, confirming that the simulated attack was fully contained within the virtual machine. The output of `/proc/1/root` further suggests that there is no unexpected namespace exposure.

2.2.4 Implications of Shared Folders

While shared folders are a common method for transferring files between a virtual machine and its host, their use inherently weakens the isolation boundary. If a container within the VM has access to a host-mounted directory, it may write or overwrite files that become visible outside the VM. This creates a deliberate escape route that must be managed with caution.

However, Houdini avoids this problem entirely by not relying on shared folders for interprocess communication or result reporting. Instead, Houdini uses a lightweight Flask-based HTTP server running on the host. When a trick completes execution inside the VM, it sends its result, such as pass/fail status or diagnostic output back to the host via an HTTP POST request over a user-mode QEMU network port.

This design has several advantages:

- It eliminates the need to mount any part of the host filesystem inside the VM.
- It decouples file I/O from test reporting, reducing the attack surface.

- It enables fine-grained control over what data leaves the VM and how it is structured.

As a result, Houdini preserves a strong isolation model while still allowing real-time, structured feedback from tests running inside the VM.

2.2.5 Conclusion

This experiment confirms that Houdini provides strong host isolation by default. Malicious trick code that attempts to write to host-accessible paths is safely confined within the virtual machine. The inclusion of a clearly defined threat model ensures that the assumptions and limits of containment are well-scoped. Furthermore, the use of a Flask-based communication channel, rather than shared folders, enhances isolation by avoiding any direct file-based interaction between the guest and host. Overall, Houdini's VM-based architecture combined with controlled, network-based result reporting offers a robust and secure foundation for container security testing.

2.3 Extensibility Evaluation

This experiment demonstrates Houdini's extensibility by adding a new custom trick and running it without modifying any of the core orchestration logic. The goal is to evaluate whether new security scenarios can be introduced through configuration alone.

To implement the new trick, a configuration file was created inside the `trick_configs/` directory. This file defines the parameter space for the new test, specifying Docker runtime flags and relevant security features. The file contained the following variables:

Listing 2.7: Escape Trick: Simulated Write to Host-Mounted Path

```
1 pid_mode = [None, "--pid=host"]
2 cap_kill = [False, True]
```

```
cap_sys_ptrace = [False, True]
privileged_opts = [False, True]
apparmor_profiles = ["apparmor=unconfined", "apparmor=docker-default
    "]
user = ["huzi"]
```

Each variable represents a dimension of the test configuration space:

- **pid_mode**: toggles between isolated and host PID namespace.
- **cap_kill**: enables or disables the `CAP_KILL` capability.
- **cap_sys_ptrace**: enables or disables the `CAP_SYS_PTRACE` capability.
- **privileged_opts**: enables or disables `--privileged` mode.
- **apparmor_profiles**: selects between `unconfined` and `docker-default` AppArmor profiles.
- **user**: specifies the user under which the container should run.

Houdini automatically loads this configuration file and generates all possible combinations of the parameters. For each configuration, Houdini builds a Docker image and executes the container with the specified flags, logging the results.

To run the new trick, the user must modify the `TRICK` variable inside constants/houdini_config.py, setting it equal to the name of the new configuration file (e.g., TRICK = "check_sys_admin"). No changes are required to Houdini's main logic or scripts.

Once the variable is set, Houdini performs the following automatically:

- Loads the new configuration file.
- Builds the associated Docker image.
- Constructs the test matrix.

- Runs each configuration inside the QEMU-based virtual machine.
- Logs the results for analysis.

This process validates Houdini's modular design and confirms that new test scenarios can be rapidly integrated by simply creating a new configuration file and updating a single variable. The minimal integration effort confirms that Houdini is scalable and extensible for diverse container security evaluations.

2.4 Version Variability Support

To evaluate Houdini's ability to operate across a broad range of runtime environments, we constructed a test matrix covering diverse combinations of Docker, runc, and Linux kernel versions. We selected representative Docker versions from three eras—early (e.g., 17.03), mid-range (e.g., 20.10), and modern (e.g., 27.0.3)—along with their corresponding runc binaries. Each version set was tested on multiple kernel versions ranging from 5.4 to 6.5.

This version variability was essential for validating Case Study 1 (CVE-2024-21626). By running Houdini's CVE trick across our version matrix, we confirmed that vulnerable runc versions (e.g., 1.0.0-rc2) allowed the exploit to succeed, while patched versions did not. This not only demonstrated Houdini's accuracy in detecting confinement violations but also verified that its test logic remains functional and portable across changes in Docker's and runc's APIs, flags, and behaviors.

Houdini's ability to build containers, run tricks, and collect output without modification across these versions highlights its value as a version-agnostic regression testing framework.

Table 2.1: Docker and runc Versions Validated with Houdini

Docker Version	**runc Version**	**Kernel Version**	**Validated**
17.03.0-ce	1.0.0-rc2	6.0	✓
17.03.1-ce	1.0.0-rc2	6.0	✓
17.03.2-ce	1.0.0-rc2	6.0	✓
17.04.0-ce	1.0.0-rc2	6.0	✓
17.05.0-ce	1.0.0-rc2	6.0	✓
17.06.0-ce	1.0.0-rc3	6.0	✓
17.06.1-ce	1.0.0-rc3	6.0	✓
17.06.2-ce	1.0.0-rc3	6.0	✓
17.09.0-ce	1.0.0-rc4+dev	6.0	✓
17.09.1-ce	1.0.0-rc4+dev	6.0	✓
17.12.0-ce	1.0.0-rc4+dev	6.0	✓
17.12.1-ce	1.0.0-rc4+dev	6.0	✓
18.03.0-ce	1.0.0-rc5	6.0	✓
18.03.1-ce	1.0.0-rc5	6.0	✓
18.06.0-ce	1.0.0-rc5+dev	6.0	✓
18.06.1-ce	1.0.0-rc5+dev	6.0	✓
18.06.2-ce	1.0.0-rc5+dev	6.0	✓
18.06.3-ce	1.0.0-rc5+dev	6.0	✓
18.09.0	1.0.0-rc5+dev	6.0	✓
18.09.1	1.0.0-rc6+dev	6.0	✓
18.09.2	1.0.0-rc6+dev	6.0	✓
18.09.3	1.0.0-rc6+dev	6.0	✓
18.09.4	1.0.0-rc6+dev	6.0	✓
18.09.5	1.0.0-rc6+dev	6.0	✓
18.09.6	1.0.0-rc6+dev	6.0	✓
18.09.7	1.0.0-rc8	6.0	✓
18.09.8	1.0.0-rc8	6.0	✓
18.09.9	1.0.0-rc8	6.0	✓
19.03.0	1.0.0-rc8	6.0	✓

Continued on next page

Table 2.1 – *continued from previous page*

Docker Version	**runc Version**	**Kernel Version**	**Validated**
19.03.1	1.0.0-rc8	6.0	✓
19.03.10	1.0.0-rc10	6.0	✓
19.03.11	1.0.0-rc10	6.0	✓
19.03.12	1.0.0-rc91	6.0	✓
19.03.13	1.0.0-rc10	6.0	✓
19.03.14	1.0.0-rc10	6.0	✓
19.03.15	1.0.0-rc10	6.0	✓
20.10.0	1.0.0-rc92	6.0	✓
20.10.10	1.0.2	6.0	✓
20.10.11	1.0.2	6.0	✓
20.10.12	1.0.2	6.0	✓
20.10.13	1.0.3	6.0	✓
20.10.14	1.0.3	6.0	✓
20.10.15	1.1.1	6.0	✓
20.10.16	1.1.1	6.0	✓
20.10.17	1.1.2	6.0	✓
20.10.18	1.1.4	6.0	✓
20.10.19	1.1.4	6.0	✓
20.10.20	1.1.4	6.0	✓
20.10.21	1.1.4	6.0	✓
20.10.22	1.1.4	6.0	✓
20.10.23	1.1.4	6.0	✓
20.10.24	1.1.5	6.0	✓
23.0.0	1.1.4	6.0	✓
23.0.1	1.1.4	6.0	✓
23.0.2	1.1.4	6.0	✓
23.0.3	1.1.5	6.0	✓
23.0.4	1.1.5	6.0	✓
23.0.5	1.1.5	6.0	✓
23.0.6	1.1.7	6.0	✓

Continued on next page

Table 2.1 – *continued from previous page*

Docker Version	**runc Version**	**Kernel Version**	**Validated**
24.0.0	1.1.7	6.0	✓
24.0.1	1.1.7	6.0	✓
24.0.2	1.1.7	6.0	✓
24.0.3	1.1.7	6.0	✓
24.0.4	1.1.7	6.0	✓
24.0.5	1.1.8	6.0	✓
24.0.6	1.1.9	6.0	✓
24.0.7	1.1.9	6.0	✓
24.0.8	1.1.11	6.0	✓
24.0.9	1.1.12	6.0	✓
25.0.0	1.1.11	6.0	✓
25.0.1	1.1.11	6.0	✓
25.0.2	1.1.12	6.0	✓
25.0.3	1.1.12	6.0	✓
25.0.4	1.1.12	6.0	✓
25.0.5	1.1.12	6.0	✓
26.0.0	1.1.12	6.0	✓
26.0.1	1.1.12	6.0	✓
26.0.2	1.1.12	6.0	✓
26.1.0	1.1.12	6.0	✓
26.1.1	1.1.12	6.0	✓
26.1.2	1.1.12	6.0	✓
26.1.3	1.1.12	6.0	✓
26.1.4	1.1.12	6.0	✓
27.0.1	1.1.13	6.0	✓
27.0.2	1.1.13	6.0	✓
27.0.3	1.1.13	6.0	✓

2.5 Performance Benchmarking

Understanding Time Metrics in Microbenchmarking

In performance benchmarking, especially in systems research and software profiling, several time-based metrics are commonly used to assess how a program or process utilizes computing resources. These include wall clock time, user time, system time, and CPU utilization. Each of these captures a different aspect of execution behavior and provides insight into how efficiently a task runs.

Wall clock time, also known as elapsed time or real time, is the total time taken from the start of a program to its completion. It includes all delays, such as waiting for I/O operations, context switches, or sleep intervals. Wall clock time is measured as the actual duration perceived by a user, typically using timestamps recorded before and after the process runs.

User time refers to the amount of CPU time spent running code in user space. This includes the program's own instructions and any library code it uses. It excludes time spent in the kernel or operating system. This metric is measured by the operating system and tracks how much CPU time was directly consumed by the application itself.

System time (or kernel time) accounts for the time spent by the CPU in the operating system kernel on behalf of the program. This includes handling system calls, managing I/O, allocating memory, and other interactions with hardware or privileged OS routines. Like user time, it is reported by the operating system's process accounting.

CPU utilization is the percentage of total available CPU time that the process actively used during its lifetime. It is computed as the ratio of the sum of user and system time to the wall clock time. High CPU utilization indicates that the process is CPU-bound, meaning it spends most of its time doing computations. Low CPU

utilization may suggest the process is I/O-bound or waiting on external events.

Together, these metrics offer a multifaceted view of a program's execution profile. They help developers identify bottlenecks, assess performance regressions, and understand whether inefficiencies stem from computation, system overhead, or external delays.

2.5.1 Microbenchmark

Making the Root Filesystem

To evaluate the performance of Houdini's build and setup pipeline, we microbenchmarked the `mkfs` stage independently. This step is responsible for creating the root filesystem used by the virtual machine and typically takes around `60.21` seconds, as measured using the `/usr/bin/time` utility. The benchmark showed a **user time** of 19.43 seconds and a **system time** of 45.97 seconds, with the job receiving approximately **63% CPU utilization**. This suggests that the operation is moderately CPU-bound but also involves significant I/O operations, such as copying files and formatting the disk image.

The `mkfs` process not only formats the filesystem but also populates it with all necessary binaries and files from the overlay directory, including the trick scripts, configuration files, and supporting assets required for execution. Importantly, this stage is only performed once per setup. Once the root filesystem is built, it can be reused across multiple Houdini runs without re-invoking `mkfs`, significantly reducing the startup time for subsequent executions. However, if a new trick is implemented or an existing one is modified, `mkfs` must be rerun to ensure that the updated trick is included in the root filesystem image.

Booting the Virtual Machine

To assess the overhead of launching Houdini's virtual machine environment, we conducted a microbenchmark of the VM startup process. The virtual machine is started using QEMU with a prebuilt Buildroot root filesystem and a custom Linux kernel. The system is configured to launch a shell and immediately power off after a fixed delay, ensuring consistency in measurement.

We measured the total wall-clock time required to boot the VM, execute a trivial command, and power down. The benchmark was performed using the `/usr/bin/time -v` utility, capturing both user and system time. The wall-clock time for this process was approximately 28.47 seconds. The user time recorded was 3.52 seconds, while the system time was 9.38 seconds. Overall, the job utilized roughly 45% of the CPU during execution.

These results indicate that the boot process introduces a moderate overhead in Houdini's workflow, primarily due to QEMU's emulation and the initialization of the Linux kernel and user space environment. Despite this, the startup time remains stable across multiple invocations, providing a consistent and reliable baseline for empirical testing. This microbenchmark reinforces the practicality of using QEMU as a platform for repeatable, isolated container security experiments.

Building the Docker Image

To evaluate the performance overhead of Docker image creation within the Houdini pipeline, we conducted a microbenchmark of the `docker build` process. The benchmark was executed using the `/usr/bin/time -v` utility to capture detailed runtime metrics, including user time, system time, wall-clock time, and CPU utilization.

The wall-clock time to complete the initial build process was measured at approximately `1 minute and 13.04 seconds`. During this time, the process consumed `0.33 seconds` of user CPU time and `0.30 seconds` of system CPU time. The CPU uti-

lization was reported at `0%`, indicating that the build process was largely I/O-bound, likely due to reading files, writing intermediate layers, and performing setup tasks for the container image.

It is important to note that this benchmark reflects the case where the Docker image did not previously exist. The initial build incurs higher latency due to downloading base layers, compiling dependencies, and writing the final image to disk. In subsequent builds where layers are cached and the image already exists locally, the build time is significantly reduced, and CPU utilization may increase due to reduced I/O waits.

These results highlight that while the initial Docker build may introduce measurable delay, especially in cold-start scenarios, this cost can be mitigated through caching and reuse of previously built images. For workflows like Houdini, where consistency and speed are critical, avoiding redundant rebuilds is key to maintaining performance.

Running a Trick in Docker

To evaluate the runtime characteristics of executing a Houdini trick, we measured the performance of the command `docker run -it 504de41abf9b`.

Despite the failure status, the resource usage remains relevant for benchmarking purposes. The user time recorded was `0.10 seconds`, and the system time was `0.07 seconds`, highlighting that the trick performed a minimal amount of computation and kernel-level operations. The total CPU utilization for the job was measured at `9%`, suggesting that the container was idle or I/O-bound for most of its execution time.

These metrics confirm that even when a trick fails, its impact on system resources is negligible, making Houdini suitable for high-throughput and low-latency security testing across many configurations.

Baseline Comparison Outside Houdini

To contextualize Houdini's efficiency, we measured a baseline workflow executed manually, without the automation Houdini provides. This baseline consisted of starting the virtual machine, building the Docker image, and executing a security trick—all performed via shell scripts without caching or orchestration.

We used `/usr/bin/time -v` to capture system-level performance metrics. The manually executed baseline took approximately **55.70 seconds** of wall-clock time, with a user time of **22.51 seconds** and a system time of **18.12 seconds**. The CPU utilization for this task was **73%**, indicating moderate computational activity with some idle time.

Compared to Houdini's streamlined execution, which achieves the same test in a fraction of the time through reuse of cached artifacts and fewer manual dependencies, this demonstrates that Houdini significantly reduces runtime overhead. In addition to speed, Houdini ensures consistent environments and eliminates the variability often introduced in manual workflows.

2.5.2 Macobenchmark

To evaluate Houdini's runtime efficiency, we measured the total time required to execute a single trick, from virtual machine boot to container execution and result collection—using Python's `time` module to record start and end timestamps. The initial execution took approximately **82.53 seconds**, which includes both the time to boot the QEMU-based virtual machine and the time required to build the Docker image from scratch. This image build step is a one-time cost, contributing significantly to the initial delay. In contrast, all subsequent runs reused the prebuilt Docker image, reducing execution time to an average of **22.02 seconds**. These measurements highlight the impact of caching and image reuse on performance. The bulk of the time in repeated executions is attributed to VM startup and trick orchestration,

while container launch and trick evaluation themselves are relatively fast. Compared to direct execution on a host system without virtualization, where tricks typically complete in under 5 seconds, Houdini introduces additional isolation overhead in exchange for improved confinement and repeatability.

Chapter 3

Case Studies

3.0.1 Case Study 1: Evaluating the accuracy of CVE-2024-21626

CVE-2024-21626 [**?**] is a container escape vulnerability in runc, the low-level container runtime used by Docker and other container engines. It allows a malicious container to escape its isolation and execute arbitrary commands on the host system.

The root cause of this vulnerability lies in how runc handles cgroups for containers, which are used to manage system resources like CPU and memory. During container creation, runc opens a file descriptor to the host's `/sys/fs/cgroup` directory to configure resource limits. This descriptor remains open throughout the container's lifecycle and is exposed inside the container via `/proc/self/fd/`, which lists all open file descriptors of the running process. The vulnerability arises when an attacker sets the working directory in the Dockerfile to point to one of these exposed descriptors, such as `/proc/self/fd/N`, where `N` refers to the specific file descriptor inherited from runc that points to the host's cgroup hierarchy. This causes the container's current working directory to resolve to a privileged host path. As a result, all subsequent relative file operations in the container are interpreted from the host's cgroup filesystem rather than the container's own root, allowing unintended access or modification of host resources.

This behavior is enabled by Linux's treatment of `/proc/self/fd/N` as dynamic symlinks to open file descriptors. When dereferenced, these symlinks resolve to the original host-side path, regardless of container filesystem restrictions. Because runc and Docker do not restrict access to inherited file descriptors or sanitize the working directory path during initialization, attackers can exploit this oversight to escape isolation. By simply changing the container's working directory to a leaked file descriptor, a malicious container can directly interact with host-managed control groups. To test the exploitability of CVE-2024-21626, we designed a Houdini trick that executes the following steps on Docker and runc versions listed in Table 3.1:

1. Set the working directory to `/proc/self/fd/<fd>`, pointing to a controlled file descriptor.

2. Attempt to break out of the container by leveraging the misconfigured working directory.

3. Validate whether commands can be executed on the host system.

The CVE for this vulnerability indicates that the affected runc versions are $\geq$ `1.0.0-rc93`, $\leq$ `1.1.11`. This suggests the vulnerability was introduced in runc version rc93 and fixed in version 1.1.12. However, our tests conducted with Houdini revealed a significant discrepancy with this assertion. Our testing showed that earlier runc versions, such as `1.0.0-rc1`, `1.0.0-rc2`, `1.0.0-rc3`, and `1.0.0-rc4` also exhibited the same vulnerability. This discovery expands the scope of the CVE by revealing that the issue existed well before version `rc93`.

This discrepancy highlights a recurring issue in vulnerability disclosure processes: CVEs often rely on assumptions from static code analysis or limited testing. Without rigorous empirical validation across historical versions, critical edge cases can be missed. In the case of CVE-2024-21626, this oversight could have allowed vulnerable

deployments relying on older runc versions to remain unpatched under the false belief that they were unaffected.

3.0.2 Case Study 2

In this evaluation, we examine whether a Docker container running as a non-privileged user with `UID 1000` can interact with and manipulate host processes that are also owned by `UID 1000`. Under a default Docker configuration, the PID namespace provides process isolation by restricting a container's visibility and access to processes outside its own namespace. However, when the container is launched with the `--pid=host` flag, it shares the host's PID namespace, allowing it to enumerate and interact with host processes. This configuration effectively bypasses one of the core isolation mechanisms provided by containers. To assess the security implications of this namespace sharing, we investigate whether process-level interactions, such as sending signals or attaching debuggers are permitted between the container and host processes when UID ownership is aligned. The testing environment for this case study is in Table 3.2.

Case study 2's results are summarized in Table **??**, where all pairs of flags were tested using Houdini, and where the cells show what functionality was possible when launching containers with the pair of flags intersecting the rows and columns of Table **??**.

Our objective here is to investigate how various security flags interact with each other, and to determine the minimum set of privileges required for a container to manipulate host processes. Specifically, we looked at the following flags:

- `--pid=host`
- `CAP_KILL`
- `CAP_SYS_PTRACE`

- `--privileged`
- `apparmor=docker-default`

The evaluation began by assessing whether a container launched with `--pid=host` could list host processes using `ps aux`. As expected, the container displayed all processes running on the host, confirming that sharing the host PID namespace fully exposes the host's process table to the container.

Following this, we tested whether a container process could send a `SIGKILL` signal to a host process owned by `UID 1000`. The container was launched with `--pid=host`, allowing it to see and interact with host processes. Despite not being granted `CAP_KILL`, the signal was successfully delivered. This is because by default, both processes share the same UID and PID namespace, and under standard Linux permission checks, a process can send signals to another process it owns by UID without requiring special capabilities. However, if the target host process is owned by a different UID (another non-root user), `CAP_KILL` is required for the signal to be delivered. That capability must also be effective in the user namespace of the target process. Similarly, signaling host processes owned by `UID 0` remains restricted even with `CAP_KILL`, unless the container process itself is also running as root.

Combining `--pid=host` with `CAP_SYS_PTRACE` enables nearly the same level of interaction with host processes as granting `--privileged` mode. Specifically, it allows a containerized process to list host processes, send signals, and attach a debugger to them, all key capabilities typically associated with privileged containers. However, unlike `--privileged`, this more targeted configuration avoids giving the container unnecessary capabilities such as access to all of the host's devices. These results demonstrate that it is unnecessary to over-privilege containers when finer-grained controls can achieve the same functional outcomes with a significantly better security posture.

When the container is confined by AppArmor's `docker-default` profile, its ability to interact with host processes becomes significantly restricted. Specifically, `docker-default` explicitly denies system calls such as `signal` and `ptrace`, which are required for sending signals and attaching debuggers to other processes. As a result, even if the container shares the PID namespace with the host and possesses the necessary capabilities, these operations will fail.

However, including the `--privileged` flag when starting the container overrides AppArmor confinement by disabling the `docker-default` profile entirely. This lifts all AppArmor-imposed restrictions, allowing the container full access to the previously restricted system calls. As a result, operations that were previously blocked, such as sending signals or attaching debuggers to host processes are now permitted, even if AppArmor would normally deny them.

It is also important to note that granting capabilities such as `CAP_KILL` and `CAP_SYS_PTRACE` alone is not sufficient for interacting with host processes unless the container shares the host's PID namespace. Without the `--pid=host` flag, the container operates in its own isolated PID namespace, meaning it cannot see or reference host PIDs at all. As a result, any attempt to send signals or attach to host processes will fail.

We note that the explanation above was derived after substantial amount of testing and analysis. The Docker documentation alone does not clarify this behavior. Instead, users are expected to understand both the Docker tooling and the underlying Linux kernel security model to correctly select these flags.

When the container shares the same user namespace as the host (or has aligned UID mappings), and the container process runs under the same UID as the host processes, it can bypass security restrictions that would normally prevent such interactions. In this case, the container may be able to perform operations like sending signals or attaching debuggers to host processes, even without elevated capabilities

like `CAP_KILL` or `CAP_SYS_PTRACE`. These capabilities, which are typically required for such actions, are not necessary when the container and host share UID mappings, as the container process is considered the same user as the host processes it targets.

This case study highlights a core strength of Houdini, which is its ability to systematically explore and identify the least privilege for a given operation. By allowing users to declaratively modify container configurations, Houdini makes it easy to test a wide range of privilege combinations in a repeatable and automated way. Instead of relying on trial and error or defaulting to the overly permissive `--privileged` mode, users can incrementally grant permissions and observe the effect of each change. Houdini thus enables security practitioners, developers, and researchers to precisely determine the minimum set of permissions required for a given trick to succeed. As a result, Houdini not only serves as a testing framework for container escapes and misconfigurations, but also as a practical tool for privilege minimization and hardening container deployments.

This case study highlights the complexity of security policies in containerized environments, highlighting that the runtime behavior of security configurations is not always straightforward or predictable. While administrators may configure a container with `--pid=host` to enable the container to monitor host processes, there is an assumption that certain operations, such as sending signals or attaching debuggers remain restricted. However, these assumptions may not hold at runtime due to the interplay of various factors such as user namespaces and UID mappings.

3.0.3 Case Study 4 - Evaluating ptrace Enforcement Across Linux Distributions

To assess the effectiveness of Linux kernel mechanisms in enforcing process isolation within containerized environments, we conducted a series of controlled experiments targeting the ptrace system call under various configurations. These experiments were

designed to evaluate how different kernel versions, Yama security module settings, and process capabilities affect the ability of a containerized process to trace other processes, both inside and outside its container boundary.

Table 3.1: runc and crun Versions Against CVE-2024-21626

Docker Version	Runtime Version	Leaked FD	Exploit Success
runc			
-	runc 0.0.9	-	✗
1.12.0	runc 1.0.0-rc1	/proc/self/fd/4	✓
17.03.0-ce	runc 1.0.0-rc2	/proc/self/fd/4	✓
17.06.1-ce	runc 1.0.0-rc3	/proc/self/fd/5	✓
17.09.0-ce	runc 1.0.0-rc4	/proc/self/fd/5	✓
-	runc 1.0.0-rc8	-	✗
-	runc 1.0.0-rc10	-	✗
-	runc 1.0.0-rc91	-	✗
20.10.17	runc 1.1.2	/proc/self/fd/9	✓
23.0.1	runc 1.1.4	/proc/self/fd/8	✓
23.0.5	runc 1.1.5	/proc/self/fd/8	✓
24.0.2	runc 1.1.7	/proc/self/fd/9	✓
24.0.5	runc 1.1.8	/proc/self/fd/9	✓
24.0.7	runc 1.1.9	/proc/self/fd/9	✓
24.0.0	runc 1.1.11	/proc/self/fd/9	✓
-	runc 1.1.12	-	✗
-	runc 1.1.13	-	✗
-	runc 1.1.14	-	✗
crun			
-	crun 0.15	-	✗
-	crun 0.17	-	✗
-	crun 1.0.0	-	✗
-	crun 1.2.0	-	✗
-	crun 1.3.0	-	✗
-	crun 1.5.0	-	✗
-	crun 1.6.0	-	✗
-	crun 1.7.0	-	✗
-	crun 1.8.0	-	✗
-	crun 1.9.0	-	✗
-	crun 1.10.0	-	✗
-	crun 1.11	-	✗
-	crun 1.12	-	✗
-	crun 1.13	-	✗
-	crun 1.14	-	✗
-	crun 1.15	-	✗
-	crun 1.16	-	✗
-	crun 1.17	-	✗
-	crun 1.18	-	✗
-	crun 1.19	-	✗
-	crun 1.20	-	✗
-	crun 1.21	-	✗

Table 3.2: Case Study 2 Testing Environment

Component	Specification
Docker Version	27.0.3
Container Runtime	`runc` 1.1.13

Table 3.3: Ability for Containers to Interact with Host Processes Across Yama Settings (Ubuntu Kernel 6.8.12)

Security Configuration	Yama Scope	--pid=host	CAP_KILL	CAP_SYS_PTRACE	--privileged	apparmor=docker-default	--pid=host + apparmor=docker-default
--pid=host	0	♢ △	♢ △	♢ △ ⋆	♢ △ ⋆	♢	♢
	1	♢ △	♢ △	♢ △ ⋆	♢ △ ⋆	♢	♢
	2	♢ △	♢ △	♢ △	♢ △	♢	♢
	3	♢ △	♢ △	♢ △	♢ △	♢	♢
CAP_KILL	0	♢ △					♢
	1	♢ △					♢
	2	♢ △					♢
	3	♢ △					♢
CAP_SYS_PTRACE	0	♢ △ ⋆					♢
	1	♢ △ ⋆					♢
	2	♢ △					♢
	3	♢ △					♢
--privileged	0	♢ △ ⋆					♢ △ ⋆
	1	♢ △ ⋆					♢ △ ⋆
	2	♢ △					♢ △
	3	♢ △					♢ △
apparmor=docker-default	0	♢					♢
	1	♢					♢
	2	♢					♢
	3	♢					♢
--pid=host + apparmor=docker-default	0	♢	♢	♢	♢ △ ⋆	♢	♢
	1	♢	♢	♢	♢ △ ⋆	♢	♢
	2	♢	♢	♢	♢ △	♢	♢
	3	♢	♢	♢	♢ △	♢	♢

Table 3.4: *

Tests run as a non-root user (UID=1000) against a host process owned by UID 1000.
♢ List Host Process △ Send Signal to Host Process ⋆ Attach Debugger to Host Process

Table 3.5: ptrace Behavior Across Distributions with Default Yama Scopes

Distribution	Kernel Version	Yama Scope (Default)	CAP_SYS_PTRACE	Result
Debian 12	6.1 (Bookworm)	1	Yes	Success
Debian 12	6.1 (Bookworm)	1	No	Fail
Debian 8	3.16 (Jessie)	0	Yes	Success
Debian 8	3.16 (Jessie)	0	No	Success
Fedora 40	6.7	2	Yes	Fail
Fedora 40	6.7	2	No	Fail
Ubuntu 24.04	6.8.12	1	Yes	Success
Ubuntu 24.04	6.8.12	1	No	Fail
Ubuntu 22.04	5.15 (LTS)	1	Yes	Success
Ubuntu 22.04	5.15 (LTS)	1	No	Fail
Ubuntu 16.04	4.4 (LTS)	0	Yes	Success
Ubuntu 16.04	4.4 (LTS)	0	No	Success
Alpine 3.19	6.1	0	Yes	Success
Alpine 3.19	6.1	0	No	Success
Arch Linux	6.8 (Rolling)	1	Yes	Success
Arch Linux	6.8 (Rolling)	1	No	Fail
CentOS 7	3.10	1	Yes	Success
CentOS 7	3.10	1	No	Fail
CentOS 6	2.6.32	0	Yes	Success
CentOS 6	2.6.32	0	No	Success